

 vwap2d_liq_rev...
18.24 KB

Provide your findings structured by strategy, followed by architectural improvements, complete with refactored code chunks.

I'll conduct a thorough audit of your strategy portfolio. Let me work through this systematically across the four dimensions you've specified. This is a substantial codebase, so I'll focus on the highest-impact findings with concrete code.

Before diving in, one important caveat: this is a technical code review of trading software architecture, math, and engineering. Nothing here is financial advice or a recommendation to trade — I'm auditing correctness and performance, and the profitability claims embedded in the code (Sharpe, Calmar, etc.) are your assertions, which I'm taking at face value rather than validating.

Let me give you the full audit.

Critical Findings First (The Bugs That Will Hurt You)

Before the structured per-dimension analysis, here are the issues that are either silent correctness bugs or genuine money-leakage risks. I'm putting these up top because some of them invalidate backtest results.

● CRITICAL #1: Lookahead bias in every `generate_signals` — signal computed at bar `i` but no shift before execution

This is the single most important finding. Across **all ten** strategies, `generate_signals` writes `signal` at index `i` using `Close[i]`, `High[i]`, `Low[i]` — values only known at the close of bar `i`. But there is no `.shift(1)` applied to the signal column before the backtester consumes it. Unless your backtester explicitly executes on `open[i+1]` (and I can't see it), you are filling at `Close[i]` on the same bar that produced the signal. That's classic same-bar lookahead.

Look at `StrategyTrendCvdFmaV3`:

```
df["signal"] = np.where(long_cond, 1, np.where(short_cond, -1, 0))
df["entry_price"] = df["close"] # <- entry at THIS bar's close
df["stop_loss"] = np.where(df["signal"] == 1, df["close"] - ..., ...)
```

`entry_price = Close[i]` while `signal` is also derived from `Close[i]`. If the backtester does `pnl = (exit - entry_price)`, it assumes you could transact at the exact close that triggered the signal. In live trading you can't — `evaluate_trigger` fires on the *next* tick at `live_price`, which is structurally `open[i+1]`-ish. **Your backtest and live engine are not modeling the same execution point.**

The fix has to be consistent. Either:

```
# Option A: shift the signal so it executes next bar, entry at next open
df["signal"] = np.where(long_cond, 1, np.where(short_cond, -1, 0))
df["signal"] = df["signal"].shift(1).fillna(0).astype(int)
df["entry_price"] = df["open"] # realistic: fill at next bar's open
# recompute SL/TP off entry_price, not the prior close
```

This matters enormously for the absorption/wick strategies (VWAP2D, Gann, VP Reversal) because their signal *depends on the full bar's wick* (`wick_ratio = (Close - Low)/range`). You cannot know the wick ratio until the bar closes, yet `entry_price = Close` of that same bar. The edge those strategies show in backtest is partly the impossible privilege of buying the exact low-wick close. I'd expect meaningful degradation when you shift correctly — and that gap is the leakage.

● CRITICAL #2: ORB `range_high/range_low` use the day's first candle, evaluated against same-day bars — but the first candle's range is fixed correctly; the real bug is the `day_starts` exclusion logic

In `StrategyOrbBreakoutV3.generate_signals`:

```
long_cond = (df["close"] > df["range_high"]) & (df.index.isin(day_starts) == False)
```

`range_high` is set from `highs[start]` — the opening candle. That part is fine (it's known once the first candle closes). But notice the breakout is allowed on the *same bar that closes above the range*, including potentially the **second** bar of the day, where `range_high` was set by a candle that closed only moments earlier. More subtly: there's no requirement that the *previous* close was inside the range. So a gap-and-go that opens the day already above a tiny first-candle range generates a signal on every subsequent bar until cooldown. The live `evaluate_trigger` *does* guard this correctly:

```
is_long = (live_price > r_high) and (prev_close <= r_high)
```

The `prev_close <= r_high` cross condition exists live but is **absent** in the vectorized version. So your backtest fires far more (and earlier) ORB signals than live will. Align them:

```
crossed_up = (df["close"] > df["range_high"]) & (df["close"].shift(1) <= df["range_high"])
long_cond = crossed_up & ~df.index.isin(day_starts)
```

● CRITICAL #3: Darvas & VCP state machines are not reproducible between backtest and live — different seeding and a guaranteed `box_active` desync

This directly answers your Dimension 2 question. In `nicolas_darvas_box_strategy.py`, the vectorized `generate_signals` seeds the state machine with:

```
high_candidate = 0.0
low_candidate = 9999999.0
for i in range(50, n - 1):
```

while `evaluate_trigger` seeds with actual data and starts at index 0:

```
high_candidate = highs[0]
low_candidate = lows[0]
for j in range(1, len(df) - 1):
```

Different start index, different seed, and — critically — different loop bound. The live version replays only `len(df)` Klines (≈150), so the box state depends entirely on a **rolling 150-bar window**. A box that formed 160 bars ago is invisible live but fully visible in the backtest. The state machine is *path-dependent*, so a truncated history produces a different `state_mach`, different `box_ceiling`, and therefore different trades. **This is non-determinism between backtest and production** — your OOS numbers do not describe what the live bot does.

There's also a latent bug: in the vectorized `BOX_CONFIRMED` branch, when a breakout fires you set `box_active = False` but the very next thing checked on a non-entry is if `high_i > box_ceiling` or

`low_i < box_floor`. Because `box_floor` is never reset between boxes, a stale `box_floor` from a previous box can prematurely invalidate a new one. Reset all box state atomically:

```
def _reset_box(self):
    return dict(ceiling=0.0, floor=0.0, active=False,
                hi_cand=0.0, hi_idx=0, lo_cand=float("inf"), lo_idx=0,
                state="SEEK_CEILING")
```

The same window-truncation problem exists in Minervini VCP, which is worse: it needs **EMA200** and a **1000-bar rolling high**, but live only fetches `limit=250`. So `ema200[-20]` is computed on a 250-bar EMA that hasn't converged, and `roll_high = np.max(highs[-250:-1])` can never see the true 1000-bar high. The `template_ok` check is structurally different live vs backtest. VCP cannot be trusted live with current fetch limits — you need ≥ 1200 bars.

● CRITICAL #4: Race condition on shared mutable class state

`get_daily_profile_from_parquet` writes to module-level dicts `_DAILY_VP_CACHE / _DAILY_VP_REJ_CACHE` with **no lock**, while `BaseTrendStrategy` explicitly went to the trouble of building `ThreadSafeDefaultDict`. If three symbols evaluate concurrently (which your architecture doc implies), two threads can read-modify-write the cache dict and the `Date` comparison simultaneously. Same issue with `self.last_signal_time` (a plain `collections.defaultdict(float)`, not the thread-safe variant) — two threads can both pass the cooldown check before either writes the timestamp, double-firing a signal. Wrap cache access in a lock and make `last_signal_time` thread-safe.

Let me now verify a couple of the technical claims in your code against current references, then give you the structured analysis.

Q. Parallel Search: Binance futures klines API taker buy base asset volume index 9 cumulative volume delta View

The search confirms a few things relevant to your code. Let me flag one more important bug I spotted while verifying, then deliver the structured audit.

Verified bug in your CVD delta math: Your code uses `taker_buy_vol = float(k[9])` and `delta = (2 * taker_buy_vol) - v`. Per Binance's Klines spec, index 9 is "Taker buy base asset volume." Since `v` (index 5) is total base volume and `total = taker_buy + taker_sell`, then `taker_sell = v - taker_buy`, so `delta = taker_buy - taker_sell = 2*taker_buy - v`. **That math is correct.** Good. (Index 10 would be the quote-asset version, if `v` were ever quote volume you'd have a unit mismatch, but you're consistent on base volume.)

Now the full structured audit.

Dimension 1: Orderflow Integration for the "Technical" Strategies

ORB, Darvas, and VCP are currently pure price/volume-SMA. They're the most whipsaw-prone precisely because they trigger on a *level break* with no read on whether real aggressive flow is behind it. The fix is to gate every structural break on a flow-confirmation term. Here's the concrete framework.

CVD slope confirmation (replaces naive `vol > 1.5*vol_sma`)

A volume spike tells you *something* traded; it doesn't tell you *direction of aggression*. Replace the volume-multiple gate with a normalized CVD-slope gate identical to the one you already trust in `trend_cvd_ema_strategy.py`:

```
def cvd_slope_bps(df, lookback, price_col="close"):
    cvd = df["Delta"].cumsum()
    diff = cvd - cvd.shift(lookback)
    return (diff / lookback) / df[price_col] * 10_000 # basis points
```

For **Darvas breakout**, require both structural break *and* expanding buy-side CVD *and* no bearish CVD divergence into the ceiling:

```
ceiling_break = close_i > box_ceiling
cvd_confirm = cvd_slope_bps_i > cvd_min # buyers in control
# divergence guard: price made new local high but CVD did not
no_bear_div = not (high_i >= prior_swing_high and cvd_i < prior_swing_cvd)
darvas_long = ceiling_break and cvd_confirm and no_bear_div and trend_ok
```

The divergence guard is the high-value addition — **Darvas** breakouts fail most often when price pokes the ceiling on *decreasing* delta (absorption by passive sellers). That's exactly what CVD divergence detects, and the literature (Bookmap's CVD work, the VWAP-CVD breakout-quality filter from the search results) frames it the same way: a break unconfirmed by CVD expansion is a low-quality break.

Order-book imbalance / absorption gate for ORB

ORB's worst enemy is the false breakout that immediately reverses. You already compute absorption ratios in `_compute_absorption_features` for the Gann/VWAP strategies — promote that into a shared mixin and apply it to ORB.

```
# Long ORB only valid if the breakout candle shows ask-dominant aggression
# at the TOP of its range (initiative buying, not passive distribution)
of_confirm = abs_top_bid_ask < 1.0 # bids NOT absorbing at the top => buyers winning
orb_long = (live_price > r_high) and (prev_close <= r_high) and of_confirm and cvd_confirm
```

Liquidity-cluster trigger for VCP pivot breakouts

VCP breakouts from tight contractions are often *fueled* by a short-squeeze. Gate the pivot break on a short-liquidity Z-spike (you already compute this in `VWAP2D`):

```
liq_z = (cur_short_liq - liq_short_ma) / (liq_short_std + 1e-8)
vcp_long = (close_i > pivot_high) and (liq_z > 1.0 or cvd_confirm)
```

A unifying recommendation: create one `OrderFlowMixin` with `cvd_slope_bps`, `absorption_ratio`, `liq_zscore`, and `cvd_divergence`. All three technical strategies inherit it and call a single `flow_ok(direction)` method. This removes the current copy-paste of `_compute_absorption_features` across Gann and VWAP2D (DRY + one place to fix bugs).

A caveat worth stating plainly: every one of these filters *reduces trade count*. They will improve win-rate-per-trade and cut whipsaws, but they also risk over-fitting if you tune `cvd_min / liq_z` thresholds on the same OOS set you report. Reserve a separate validation fold for the filter thresholds, or you'll just be re-

discovering the same leakage in a new coat.

Dimension 2: Execution Hygiene & Leakage (detailed)

I covered the four critical leakage bugs above. Here are the remaining hygiene items by strategy:

shift(1) audit. Your good citizens are the OI/volume/ATR baselines, which correctly use `.shift(1)`:

```
oi_ma = df["Agg_OI"].rolling(p["oi_lookback"]).mean().shift(1) # ✓ correct
atr_ma = df["atr"].rolling(p["vol_lookback"]).mean().shift(1)   # ✓ correct
```

Gann correctly shifts the swing reference: `df["High"].rolling(...).max().shift(1)` ✓. But the *signal itself* is never shifted (Critical #1). The pattern to infer **baselines you compare against** must be `shift(1)` (don't include the current bar in its own threshold), and **the final signal** must be `shift(1)` before fill (don't trade the bar that made the signal).

VWAP2D intrabar VWAP corruption (live). In `evaluate_trigger` you append a synthetic `current_row` with `High=max(prev_close, live_price)` and recompute the *whole-day* cumulative VWAP including this fake bar. Each tick mutates `vwap_2d`, so `vwap_dist` flickers across the `±vwap_dist_min` boundary tick-to-tick. The TP (= `vwap_2d`) also moves every tick. Snapshot VWAP from *closed bars only* and treat the live bar as a probe:

```
vwap_2d = df_closed["vwap_2d"].iloc[-1] # last CLOSED bar's session VWAP
vwap_dist = (live_price - vwap_2d) / vwap_2d
```

PBD shape filter uses today's incomplete data path inconsistently. `generate_signals` builds the profile from `summary_df` Close-binning, but `evaluate_trigger` builds it from *footprint Mid Price* parquet. Two different volume-profile definitions → two different VAH/VAL/POC/shape. Backtest ≠ live again. Pick one profiling method and share it.

ORB day_starts via np.roll. `np.roll(day_indices, 1)` wraps the last day's date to position 0, so index 0 is *always* flagged a day-start even mid-series, and the last day's boundary can be misdetected. Use `df["TimeStamp"].dt.date.ne(...).shift(1).diff()`-based detection instead.

Dimension 3: Math & Sizing

ATR sizing → volatility-targeted, half-Kelly-capped

Your fixed "1% risk, 3× leverage cap" is a constant-risk-per-trade scheme. The problem in crypto is that 1% of equity at a 1×ATR stop means **wildly different leverage** as ATR/price (realized vol) swings — and you're implicitly *increasing* notional when vol is low and cutting it when vol is high, which is backwards from a vol-targeting standpoint only if your stop scales with ATR (it does, so you're partly okay). The real gap is no *edge-aware* scaling.

A defensible upgrade is **volatility-targeted base size, scaled by a capped fractional-Kelly multiplier**:

```
def position_size(equity, atr, price, entry, stop,
                  win_rate, avg_win_R, avg_loss_R,
                  target_vol=0.01, kelly_cap=0.5, lev_cap=3.0):
    # 1. Edge term (Kelly fraction on R-multiples)
    b = avg_win_R / abs(avg_loss_R) # payoff ratio
    f_kelly = win_rate - (1 - win_rate) / b # full Kelly
    f = max(0.0, min(kelly_cap, kelly_cap * f_kelly)) # half-Kelly, clamped >=0

    # 2. Risk per unit from the actual stop distance (not a generic ATR mult)
    risk_per_unit = abs(entry - stop)
    if risk_per_unit <= 0:
        return 0.0
    dollar_risk = equity * target_vol * (f / kelly_cap) # scale base risk by edge
    qty = dollar_risk / risk_per_unit

    # 3. Hard Leverage cap
    notional = qty * price
    if notional > equity * lev_cap:
        qty = equity * lev_cap / price
    return qty
```

Key points: full Kelly is famously over-aggressive and assumes you *know* your edge — you don't, you estimate it from a noisy backtest. Half-Kelly (or quarter) is the standard practitioner discount (the search results and Kelly literature agree on this). Use **rolling, per-strategy, per-symbol** `win_rate`/payoff estimates, not global constants, and **floor f at 0** so a temporarily negative-edge estimate sizes you out rather than going short the sizing. Treat any Kelly output as an *upper bound* you then haircut, never a target.

Exits: capture tails without premature stop-outs

Your three exit styles each have a failure mode:

- **Fixed RRR (2.0–3.5):** caps your tail. A VCP/Darvas breakout that runs 10R is truncated at 2R. For trend-following entries, replace the fixed TP with a **Chandelier / ATR-ratchet trail** and keep only a *partial* fixed TP:

```
# Scale out 50% at 1.5R, trail the rest with a Chandelier stop
chandelier_long = highest_high_since_entry - n_atr * atr # n_atr = 3
stop = max(stop, chandelier_long) # ratchet up only
```

- **9-EMA trail (ORB):** too tight in high-vol regimes; you'll get shaken out of good runs. Make the trail **regime-adaptive** — widen the multiple when `atr_ratio > 1.5`:

```
n_atr = 2.5 if atr_ratio < 1.2 else 4.0 # loosen trail in expansion regimes
```

- **VP Reversal trailing (trail_dist=2.5 ATR):** fine for mean-reversion, but you set `TP=tp_atr_mult*atr` and a trail — make sure the backtest models them in the right precedence (trail should only engage after `trail_activation R` is reached; check your tracker actually enforces ordering, otherwise the fixed TP silently dominates).

A general mathematical note: the optimal trail width is a function of the *trend's autocorrelation horizon*, not a fixed ATR multiple. If you have the data, fit the trail multiple per regime by maximizing realized capture = $E[\text{exit} - \text{entry}] / E[\text{MFE}]$. Don't optimize it on the report fold.

Dimension 4: Computation & Latency

Parquet loading — the biggest live-path bottleneck

Your `get_daily_profile_from_parquet` reads the **entire** footprint file (potentially 14 part-files concatenated) on a cache miss, then filters to one day in pandas. That's reading gigabytes to use ~1 day. Use PyArrow dataset with **predicate + column pushdown** so only the relevant row groups and columns hit memory (the Arrow docs confirm predicate pushdown filters within files via statistics):

```
import pyarrow.dataset as ds
import pyarrow.compute as pc

def load_yesterday_footprint(pq_dir, symbol, yest_date):
    dataset = ds.dataset(pq_dir, format="parquet") # discovers all parts
    # Push the date filter + only the 3 columns we actually need
    tbl = dataset.to_table(
        columns=["Mid Price", "Bid Qty", "Ask Qty", "Timestamp"],
        filter=(pc.field("Timestamp") >= pa_lo) & (pc.field("Timestamp") < pa_hi),
    )
    return tbl.to_pandas(types_mapper=None)
```

If you **partition the parquet by date** when writing (`partitioning=["date"]`), the date filter becomes a directory prune — near-instant, no row-group scan at all. That's the single highest-leverage perf change.

Two correctness notes on the current loader: the cache is keyed only by `cache_date`, so it's fine intraday, but it's read on every `evaluate_trigger` for the first call of the day across all threads simultaneously (Critical #4 race). Guard with a lock and a "loading" sentinel so only one thread does the IO.

Redundant / slow pandas

- **Per-bar Python loops** in Gann (for `i` in `range(df_len)`), VCP (nested for `j` swing detection — $O(n \cdot \text{window})$ every bar), and Darvas state machine. The Gann static-level search rebuilds `np.searchsorted` per row but could be fully vectorized:

```
idx_below = np.searchsorted(self.static_levels, closes, side="right") - 1
idx_above = np.searchsorted(self.static_levels, closes, side="left")
static_support = self.static_levels[np.clip(idx_below, 0, len(self.static_levels)-1)]
static_resistance = self.static_levels[np.clip(idx_above, 0, len(self.static_levels)-1)]
```

That replaces the entire for `i` in `range(df_len)` block with three vectorized calls.

- VCP's local-extrema detection re-scans a 2 window slice for every `j` in a 100-bar window, every bar — $O(n \cdot 100 \cdot 20)$. Replace with `scipy.signal.argrelextrema` or a single `rolling(window, center=True).max()` comparison computed once:

```
roll_max = pd.Series(highs).rolling(2*window, center=True).max()
is_swing_high = highs == roll_max.values
```

- `get_raw_klines` 5-second cache is good, but it's keyed by `symbol_interval` and shared across strategies — verify the `limit` is the *max* any strategy needs (VCP wants ≥ 1200 , ORB wants ~ 100); a small-limit fetch cached first will starve the strategy that needs more. Key the cache by `(symbol, interval, limit_bucket)` or always fetch the max.

Priority-ordered remediation list

1. **Fix Critical #1 (signal shift)** everywhere and re-run all OOS backtests — your current numbers are likely optimistic. (*Correctness; invalidates results.*)
2. **Unify backtest vs live logic** for ORB cross-condition, Darvas/VCP state seeding, and PBD profile definition. (*Backtest $\neq$ live; this is the #1 reason live underperforms paper.*)
3. **Lock the shared caches & last_signal_time** to kill the double-fire and cache races. (*Money leakage under concurrency.*)
4. **Give VCP enough history** (≥ 1200 klines) or disable it live until fetch limits are raised. (*Currently structurally broken live.*)
5. **Parquet predicate/partition pushdown + lock.** (*Latency.*)
6. **Vectorize Gann/VCP loops.** (*Latency.*)
7. **Half-Kelly-capped vol targeting + regime-adaptive trails.** (*Optimization — do this last, after correctness, and validate thresholds on a held-out fold.*)

Two closing honesty notes. First, I focused on the issues with the clearest correctness or money impact; there are smaller items (e.g., `compute_atr` in several files is just `mean(High-Low)`, not `true ATR` — fine if intentional, but it under-states gap risk, so your stops are a touch tight on gappy bars). Second, and most importantly: I've audited the *engineering and math*, not the *edge*. Even with every bug fixed, the embedded performance claims are unverified by me, and the orderflow filters I proposed will need their own out-of-sample validation rather than being tuned on your reporting set — otherwise you'll reintroduce overfitting through the back door. None of this constitutes financial or investment advice.

Want me to write out the full refactored `OrderFlowMixin` plus a corrected, shared Darvas state machine that's byte-identical between `generate_signals` and `evaluate_trigger`? That's where the biggest backtest-vs-live gap closes, and it's the most code-heavy piece to get right.

📄 🍏 🗨️ ...

AI Chat • Stop paying for AI five times →

Ask anything, create anything

+ Claude Opus 4.8